SOLID принципи и шаблони за дизайн.

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

SOLID принципи. Примери за шаблони - singleton, factory, adapter, strategy.

Какво са SOLID принципите?

SOLID е набор от 5 принципа за обектно-ориентиран дизайн, целящи: по-лесна поддръжка, разширяемост, по-малко грешки, по-добра архитектура.

Принципи:

S – Single Responsibility

O – Open / Closed

L – Liskov Substitution

I – Interface Segregation

D – Dependency Inversion

Single Responsibility principle

Един клас → една отговорност → една причина за промяна

❌ Лош пример:

```
class Student {
public:
    void calculateGrade();
    void saveToFile();
};
```

✅ Добър:

```
class Student { void calculateGrade(); };
class StudentRepository { void save(const Student&); };
```

Open/Closed principle

Идея:Класовете са:отворени за разширяване и затворени за модификация

❌ if / switch за типове

Типичен проблем:

```
if (type == CIRCLE) ...

else if (type == RECTANGLE) ...
```

➡ Всеки нов тип = нов if

✅ Полиморфизъм

Пример

```
class Shape {

public:

    virtual double area() const = 0;

};

class Circle : public Shape {

    double r;

public:

    double area() const override { return 3.14 *
r * r; }

};

class Rectangle : public Shape...
```

Liskov substitution principle

Определение:

Обект от наследен клас трябва напълно да замества базовия.

Проблемът със Square / Rectangle:

Rectangle обещава: ширина и височина са независими

Square нарушава това обещание

❗ LSP е за поведение, не за синтаксис

Практически сигнал за проблем:

if (dynamic_cast…)

проверки за тип

изключения в override

Правилен пример

```
class Shape {
public:
    virtual int area() const = 0;
    virtual ~Shape() = default;
};
class Rectangle : public Shape
class Square : public Shape,
```

а НЕ

```
class Square : public Rectangle
```

Interface Segregation Principle

Пример

```
class Worker {
public:
    virtual void work() = 0;
    virtual void eat() = 0;
    virtual ~Worker() = default;
};
```

Проблем

```
class Human : public Worker {
public:
    void work() override {
        // работи
    }

    void eat() override {
        // яде
    }
};
class Robot : public Worker {
public:
    void work() override {
        // работи
    }
    void eat() override {
        // ❌ роботите не ядат
        throw std::logic_error("Robot
cannot eat");
    }
};
```

Грешка

Robot е принуден да имплементира eat()
появяват се празни методи или изключения
клиентите зависят от ненужни функции
➡ ISP е нарушен

Решение

```
class Workable {
public:
    virtual void work() = 0;
    virtual ~Workable() = default;
};
class Eatable {
public:
    virtual void eat() = 0;
    virtual ~Eatable() = default;
};
```

Проблеми и решения

❌ Проблеми:

интерфейси с много методи

override-и с празно тяло

throw NotImplemented

клас „знае твърде много“

✅ Добра практика:

интерфейси с ясна роля

често имена завършващи на -able

Какво е Dependency Inversion Principle?

Модулите от високо ниво не трябва да зависят от модулите от ниско ниво.И двата вида модули трябва да зависят от абстракции.
Абстракциите не трябва да зависят от детайли.Детайлите трябва да зависят от абстракциите.
Пример:

```
class FileLogger {
public:
    void log(const std::string& msg) {
        std::cout << "File: " << msg << std::endl;
    }
};

class OrderService {
    FileLogger logger;   // ❌ директна зависимост
public:
    void createOrder() {
        logger.log("Order created");
    }
}; //OrderService знае точния клас, не можем лесно да използваме
ConsoleLogger, DBLogger
```

Правилен дизайн

```
class Logger {
public:
    virtual void log(const std::string& msg) = 0;
    virtual ~Logger() = default;
}; //интерфейс
class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        std::cout << "File: " << msg << std::endl;
    }
};

class OrderService {
    Logger& logger;   // ✅ зависимост от абстракция
public:
    OrderService(Logger& l) : logger(l) {}

    void createOrder() {
        logger.log("Order created");
    }
};
```

Какво са шаблоните за дизайн?

Шаблоните за дизайн (Design Patterns) са утвърдени, доказани решения на често срещани проблеми при проектиране на софтуер.

Те не са готов код, а описание на идея / структура, която може да се приложи в различни езици и проекти.

Защо са нужни шаблоните за дизайн?

При разработка често се сблъскваме с едни и същи проблеми: как да създаваме обекти гъвкаво, как да намалим зависимостите,как да разширяваме код без да го чупим, как да организираме комуникацията между обекти

👉 Design patterns предлагат стандартен начин да решим тези проблеми

🔍 Какво НЕ са шаблоните за дизайн?

❌ не са библиотеки

❌ не са framework-и

❌ не са конкретна имплементация

❌ не са “магия”

Те са:

✔ са архитектурни идеи

✔ са езиково независими

✔ се адаптират към конкретния проект

Creational Design Patterns

Фокусират се върху създаването на обекти.

Примери:

Singleton – един екземпляр

Factory / Abstract Factory – създаване без конкретен тип

Prototype – клониране

Builder – поетапно изграждане

Пример със **Singleton** вдясно:

```
class Logger {
public:
    static Logger& instance() {
        static Logger instance;   // създава се само веднъж
        return instance;
    } //C++ 11
    void log(const std::string& msg) {
        std::cout << msg << std::endl;
    }
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
private:
    Logger() = default;
};
```

Factory Design Pattern

Клиентът не знае точния клас, който се създава – работи с интерфейс.

Училището работи с различни видове студенти: редовен ученик, дистанционен ученик, обменен ученик

❌ Не искаме School да знае конкретните класове.

```
class Student {

public:

    virtual void attendClass() const = 0;

    virtual std::string type() const = 0;

    virtual ~Student() = default;

};
```

```
class RegularStudent : public Student {

public:

    void attendClass() const override {

        std::cout << "Student attends class in
person\n";}

    std::string type() const override {

        return "Regular";}

}; }; //class OnlineStudent : public Student {

class StudentFactory {

public:

    static std::unique_ptr<Student> createStudent(const
std::string& kind) {

        if (kind == "regular")

            return std::make_unique<RegularStudent>();

        else if (kind == "online")

            return std::make_unique<OnlineStudent>();

        return nullptr;

    }

};
```

Structural Design Patterns

Структурни (Structural Patterns)

Цел: Организират връзките между класове и обекти, за да се постигне по-добра повторна употреба и гъвкавост.

Примери:Adapter, Bridge, Composite,Decorator, Facade…

Adapter - позволява несъвместими интерфейси да работят заедно, като „превежда“ интерфейса на един клас към очаквания от клиента.

```
class Student {
public:
    virtual std::string getName() const = 0;
    virtual int getGrade() const = 0;
    virtual ~Student() = default;
};
```

Стар клас:

```
class LegacyStudent {
    std::string fullName;
    int mark;
public:
    LegacyStudent(const std::string& name, int m)
        : fullName(name), mark(m) {}
    std::string name() const { return fullName; }
    int markValue() const { return mark; }
};
```

Adapter пример

```
class StudentAdapter : public Student {
    LegacyStudent& legacyStudent;
public:
    StudentAdapter(LegacyStudent& ls) :
legacyStudent(ls) {}
    std::string getName() const override {
        return legacyStudent.name();
    }
    int getGrade() const override {
        return legacyStudent.markValue();
    }
};
```

Пример за използване:

```
#include <iostream>
class School {
public:
    void printStudentInfo(const Student& student) const
{}
};
int main() {
    LegacyStudent oldStudent("Ivan Petrov", 5);
    StudentAdapter adaptedStudent(oldStudent);
    School school;
    school.printStudentInfo(adaptedStudent);
    return 0;
}
```

Behavioural Patterns

Цел: Определят взаимодействието и комуникацията между обекти, управление на отговорностите и алгоритмите.

Примери: Observer, Strategy, Command, Iterator, Visitor

Strategy pattern - позволява да сменяме алгоритъм по време на изпълнение,без да променяме клиента (класа, който го използва).

```
class GradingStrategy {

public:

    virtual double calculateGrade(double exam,

                                  double project)
const = 0;

    virtual ~GradingStrategy() = default;

};
```

Пример за използване:

```
class ExamOnlyStrategy : public GradingStrategy {

public:

    double calculateGrade(double exam, double) const
override {

        return exam;

    }

};

class ProjectOnlyStrategy : public GradingStrategy {

public:

    double calculateGrade(double, double project) const
override {

        return project;

    }

};
```

Strategy пример

```
class Student {
    GradingStrategy* strategy;   // зависимост от абстракция
public:
    Student(GradingStrategy* s) : strategy(s) {}

    void setStrategy(GradingStrategy* s) {
        strategy = s;
    }

    double finalGrade(double exam, double project) const {
        return strategy->calculateGrade(exam, project);
    }
};

int main() {
    ExamOnlyStrategy exam;
    ProjectOnlyStrategy project;
    Student student(&exam);
    std::cout << student.finalGrade(5.50, 4.00) << std::endl;
    student.setStrategy(&mixed);
    std::cout << student.finalGrade(5.50, 4.00) << std::endl;
    return 0;
}
```

Въпроси?

Благодаря за вниманието!